

Heterogeneous Graph-Attention Reinforcement Learning for Indian Equity Portfolio Allocation

A Technical Report on Architecture, Implementation, and the Path to a Working Baseline

Trading Bot Project — Milestones M0 through M6

April 2026

Abstract

We present the design and implementation of a deep reinforcement learning system for daily portfolio allocation over a 163-stock Indian equity universe (NIFTY 50 augmented with sectoral leaders). The agent is a heterogeneous graph attention network operating on stock and sector nodes, fed by a temporal convolutional encoder over a 60-day feature window, and trained end-to-end with Proximal Policy Optimization (PPO). We document the full pipeline from raw yfinance bars through walk-forward training, including a Postgres-backed paper-broker infrastructure stub for downstream evaluation. Beyond architecture, we report on a sequence of non-trivial bugs discovered during initial training runs — a *Differential Sharpe* reward with zero expected value at steady state, raw input features spanning eleven orders of magnitude in scale, and a Python-loop bottleneck in the environment that disguised itself as a GPU-bound problem. After fixing each, the simpler intra-sector graph variant achieves the first positive training Sharpe in the project’s history. We close with a roster of pre-staged improvements for the next training round and a path to walk-forward evaluation.

Contents

1	Introduction	3
1.1	Goal	3
1.2	Non-goals (v1)	3
1.3	Design Principles	3
1.4	Milestone Roadmap	3
1.5	Hardware and Software Stack	3
2	Data Pipeline	4
2.1	Universe Construction	4
2.2	Sources	4
2.3	Storage Layout	5
2.4	Alignment and Masking	5
2.5	Feature Engineering	5
2.6	Train / Validation / Test Splits	5
3	Trading Environment	6
3.1	Architecture	6
3.2	Observation Space	6
3.3	Action Space and the Masked Softmax Projection	6
3.4	Reward Functions	7
3.5	Cost Model	7
3.6	Slippage Model	8
3.7	Episode Termination	8

4	Model Architecture	8
4.1	Feature Normalizer	8
4.2	TCN Encoder	9
4.3	Heterogeneous Graph Attention Network	9
4.3.1	A Concrete Six-Stock Example	9
4.3.2	Batching and Masking	10
4.3.3	Ablation Variants	11
4.4	Actor and Critic Heads	11
4.4.1	Actor Head	11
4.4.2	Critic Head	11
4.5	Parameter Budget	11
5	Reinforcement Learning Algorithm	12
5.1	Why PPO	12
5.2	The PPO Objective	12
5.3	Generalized Advantage Estimation	12
5.4	Reward Normalization	13
5.5	Mixed-Precision Training	13
5.6	Target-KL Early Stopping	13
5.7	Algorithm Summary	13
6	A Step-by-Step Walkthrough	13
7	The Debugging Journey	15
7.1	Run-by-Run Status	15
7.2	Bug 1: Stochastic Operations During Rollout	15
7.3	Bug 2: BF16 / FP32 Precision Mismatch	16
7.4	Bug 3: DifferentialSharpe Reward Has Zero Expected Value	16
7.5	Bug 4: Feature Scale Catastrophe	16
7.6	Bug 5: Environment Was the Wall-Clock Bottleneck	17
7.7	Architectural Hardening (Beyond Bugs)	17
8	Empirical Results	17
8.1	Baselines	17
8.2	r6 Results In Progress	17
8.3	Why GNN_v1 Underperforms GNN_intra	18
9	Hyperparameter Reference	18
9.1	PPO Hyperparameters	18
9.2	Model Hyperparameters	19
9.3	Environment Hyperparameters	19
10	Future Work	19
10.1	r7: Pre-staged Improvements	19
10.2	M7: Walk-Forward Training	20
10.3	M8–M10: Paper Broker, Live Stub, End-to-End Report	20
10.4	Beyond M10: Possible Extensions	20
11	Lessons Learned	20

1 Introduction

1.1 Goal

Build a local, paper-trading-capable reinforcement learning system that learns a daily portfolio-allocation policy over the Indian equity universe (NSE), using a graph-aware neural network over intra-sector and inter-sector relationships, trained on roughly ten years of historical data, with a Postgres-backed simulated broker and stubbed hooks for live Zerodha Kite trading.

The agent decides, each trading day, what fraction of capital to hold in cash and what fraction to allocate to each of N tradeable stocks, subject to long-only and per-name weight cap constraints. Quality is measured by net-of-cost Sharpe ratio against representative baselines (equal-weight rebalanced, buy-and-hold index, momentum top-K, 60/40 cash, and uniform random).

1.2 Non-goals (v1)

- Real-money live trading (interfaces only; disabled by default).
- Sub-minute or tick-level data (daily bars; intraday is a future variant).
- LLM-in-the-loop at inference time (sentiment, if used at all, is offline).
- Multi-user / multi-account / hardened production deployment.

1.3 Design Principles

1. **Strict temporal separation.** Features at day t use only data with timestamp $\leq t - 1$ for actions executed on day t . Actions are placed at the open of day t ; portfolio is marked-to-close.
2. **Universe as of date.** Historical universe snapshots prevent survivorship bias — selecting “today’s top N per sector” and backtesting on ten years is selecting winners after the fact.
3. **Mask, don’t impute.** Stocks not yet listed (or delisted) are masked out of the observation and action space for the relevant dates. No zero-filling of non-existent tickers.
4. **Broker-agnostic.** A Broker ABC with a paper implementation backed by Postgres, and a Zerodha Kite stub. The live adapter is behind a flag that is off by default.
5. **Reproducibility.** Seeds pinned, data snapshotted with a SHA256 hash, `uv lock` for dependencies, MLflow for runs, dockerized Postgres.
6. **Baselines first.** Buy-and-hold NIFTY 50, equal-weight monthly rebalance, momentum top-K, and 60/40 are implemented and measured *before* the RL agent. The RL agent has to beat them net of costs.
7. **One end-to-end path before specialization.** A minimal flat-feature PPO run must work on the full pipeline before GNN, sentiment, or hierarchical policies are added.

1.4 Milestone Roadmap

1.5 Hardware and Software Stack

Development is performed on macOS (Apple Silicon, MPS backend); training runs are executed on a Linux server with CUDA 12.8 and an RTX 5090 (Blackwell). Software stack: Python 3.12, PyTorch (BF16 autocast), `torch_geometric` for GAT layers, Polars for tabular data, NumPy/numba-style numpy-only env code, Hydra for configuration, MLflow for experiment tracking.

Table 1: Project milestones and current status as of April 26, 2026.

Milestone	Scope	Status
M0	Repo skeleton, uv, ruff, mypy, pytest, dockerized Postgres + MLflow	Complete
M1	Database schema; SQLAlchemy 2.0 models; Alembic migration	Complete
M2	Data ingestion; yfinance source; Kaggle stub; Zerodha stub	Complete
M3	Alignment + features; train/val/test panels with SHA256 sidecars	Complete
M4	PanelTradingEnv + cost model + reward + 5 baselines	Complete
M5	MLP policy + PPO; CleanRL-style single-file trainer	Complete
M6	Heterogeneous GNN with sector hierarchy + 3 ablation configs	Complete
M7	Walk-forward training	Pending
M8	Paper broker; T+1 settlement; ledger reports	Pending
M9	Zerodha adapter stub	Pending
M10	End-to-end report; attention heatmaps; significance tests	Pending

2 Data Pipeline

2.1 Universe Construction

The universe consists of 163 NSE-listed equities organized into 8 sectors of roughly equal size. The sector taxonomy was constructed manually from sectoral indices and is fixed across the entire training period. Table 2 summarises.

Table 2: Sector composition of the 163-stock universe. Sector IDs are 1-indexed; ID 0 is reserved for “unclassified” (used internally only — no stock in the universe falls into this bucket).

Sector	ID	Representative tickers
Banking + NBFC + Insurance	1	HDFCBANK, ICICIBANK, KOTAKBANK, BAJFINANCE, SBIN, ...
Information Technology	2	TCS, INFY, HCLTECH, WIPRO, LTIM, MPHASIS, ...
Pharma + Healthcare	3	SUNPHARMA, DRREDDY, CIPLA, DIVISLAB, APOLLOHOSP, ...
Automotive	4	MARUTI, M&M, BAJAJ-AUTO, EICHERMOT, TATAMOTORS, ...
Oil, Gas & Power	5	RELIANCE, ONGC, BPCL, NTPC, POWERGRID, COALINDIA, ...
Metals & Mining	6	TATASTEEL, JSWSTEEL, HINDALCO, COALINDIA, NMDC, ...
Consumer + Retail	7	ITC, HINDUNILVR, NESTLEIND, BRITANNIA, ASIANPAINT, ...
Capital Goods + Defence + Cement	8	LT, SIEMENS, ABB, BHEL, HAL, ULTRACEMCO, ...

2.2 Sources

Three concrete data sources implement a common `MarketDataSource` Protocol:

- `YFinanceSource` — primary source; per-ticker cache at `data/raw/yfinance/<ticker>/<start>_<end>.parquet` with exponential backoff and `auto_adjust=True` (so close prices are split- and dividend-adjusted).
- `KaggleSource` — backfill source for delisted or pre-IPO tickers, loaded from `data/raw/kaggle/<TICKER>.parquet`.
- `ZerodhaSource` — interface stub for live data; raises `NotImplementedError`.

2.3 Storage Layout

OHLCV is partitioned Hive-style at

`data/ohlcw/year=YYYY/ticker=TICKER.parquet` with idempotent writes (rerunning `scripts/fetch_data.py` is a no-op on already-cached date ranges).

2.4 Alignment and Masking

`src/trader/data/alignment.py` produces a dense ($date \times ticker$) panel using the NSE trading calendar derived from the union of observed dates across all tickers. The key invariant:

A ticker that was first listed in 2017 has `is_tradeable = False` for all dates in 2014–2016. Its rows still exist in the panel, but with `adj_close = 0` sentinel and `is_tradeable = False`, so models and the env must consult the mask to know which stocks are actionable on each date.

Forward-fill is applied with a strict `limit=1` (single-day gaps only). On gap days, volume is set to zero. Multi-day gaps surface as `is_tradeable = False` and propagate that flag.

2.5 Feature Engineering

Fifteen features are computed strictly from backward-looking windows. All formulas use positive-shift operators only; a static lint check rejects any `shift(-k)` or `rolling(...).apply(...)` that could leak future information. Features are listed in Table 3.

Table 3: Feature set and observed scale (std on tradeable rows of the train panel, 2014–2021). Note the eleven orders of magnitude range — the source of the catastrophic learning failure detailed in §7.

Feature	Definition	Mean	Std
<code>log_return_1d</code>	$\log(p_t/p_{t-1})$ on <code>adj_close</code>	7.2e-4	2.3e-2
<code>log_return_5d</code>	$\log(p_t/p_{t-5})$	3.6e-3	5.4e-2
<code>log_return_20d</code>	$\log(p_t/p_{t-20})$	1.5e-2	1.1e-1
<code>realized_vol_20d</code>	20-day std of <code>log_return_1d</code> $\cdot \sqrt{252}$	3.3e-1	1.7e-1
<code>realized_vol_60d</code>	60-day std $\cdot \sqrt{252}$	3.4e-1	1.4e-1
<code>rsi_14</code>	Wilder’s RSI, $\alpha = 1/14$	5.2e+1	1.2e+1
<code>macd</code>	EMA(12) – EMA(26) on <code>adj_close</code>	5.9	1.1e+2
<code>macd_signal</code>	EMA(9) of <code>macd</code>	6.0	1.0e+2
<code>macd_hist</code>	<code>macd</code> – <code>macd_signal</code>	-5.5e-2	3.1e+1
<code>bbw_20</code>	Bollinger band width, $4\sigma_{20}/\mu_{20}$	1.5e-1	1.1e-1
<code>z_close_20</code>	$(p_t - \mu_{20})/\sigma_{20}$	1.6e-1	1.3
<code>volume_z_20</code>	20-day volume z-score	1.7e-4	1.0
<code>dollar_volume_20</code>	20-day mean of $p_t \cdot v_t$ (INR)	1.3e+9	2.4e+9
<code>atr_14</code>	Wilder’s average true range	3.6e+1	1.4e+2
<code>beta_nifty_60d</code>	60-day rolling β vs $\hat{N}SEI$ (default 1 if absent)	1.0	0.0

2.6 Train / Validation / Test Splits

Splits use one-month *purge gaps* between regimes to discourage label leakage through serial correlation:

- **Train:** $\leq 2021-12-31$ (1972 trading days, 2014–2021)
- **Val:** 2022-02-01 to 2022-12-31 (228 days)

- **Test:** 2023-02-01 to end (held out until M10)

Each split is written to `data/panels/{train,val,test}.parquet` with a SHA256 sidcar and a row in `market.dataset_versions` for provenance. A deterministic rerun of `scripts/build_features.py` produces byte-identical outputs.

3 Trading Environment

3.1 Architecture

`PanelTradingEnv` is a `Gymnasium Env` that steps one trading day at a time over a windowed slice of the panel. Per-episode, a uniformly-random `start_idx` is sampled from $[\text{lookback}, T - \text{episode.length} - 1]$ so the agent sees diverse market regimes within a single training run.

Figure 1 shows the per-step data flow.

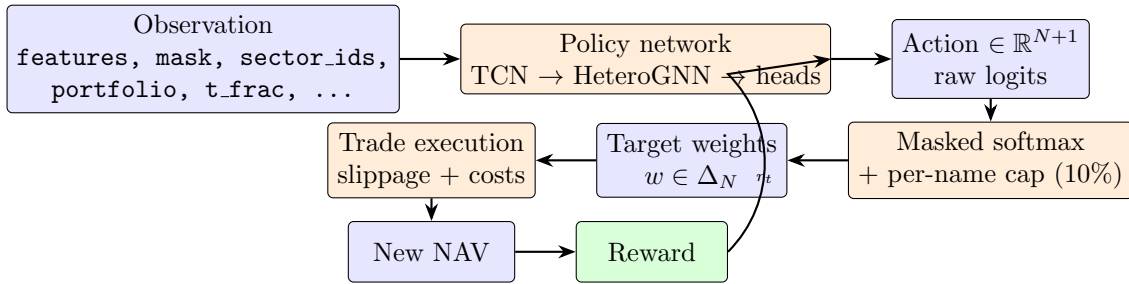


Figure 1: Per-step data flow inside `PanelTradingEnv`. The policy outputs raw logits which are projected to the simplex with a per-name cap; trades are filled at the open with ATR-scaled slippage; portfolio is marked-to-close; reward is computed from the resulting log return.

3.2 Observation Space

The observation is a `Gymnasium Dict` with the following keys (Table 4). Note that the last three fields (`recent_return_1d`, `recent_vol_20d`, `nav_log_progress`) were added during the r6 critic-enrichment pass and are consumed only by the critic head.

Table 4: Observation dictionary. $L = 60$ is the lookback, $N = 163$ is the universe size, $F = 15$ is the feature count.

Key	Shape	Meaning
<code>features</code>	(L, N, F)	Feature panel ending at $t - 1$
<code>mask</code>	$(N,)$	<code>is_tradeable</code> on day t
<code>sector_ids</code>	$(N,)$	1-indexed sector assignment (0 = unknown)
<code>portfolio</code>	$(N + 1,)$	Index 0 = cash weight; $1 \dots N$ = equity weights
<code>cash</code>	$()$	Cash position in INR
<code>nav</code>	$()$	Current net asset value
<code>t_frac</code>	$()$	Episode progress $\in [0, 1]$
<code>recent_return_1d</code>	$()$	Last day's portfolio log return
<code>recent_vol_20d</code>	$()$	Std of last 20 daily log returns
<code>nav_log_progress</code>	$()$	$\log(\text{NAV}_t / \text{NAV}_0)$

3.3 Action Space and the Masked Softmax Projection

The action space is `Box(-10, 10, (N + 1,))` — raw logits for cash and each of the N equity slots. The env applies a masked softmax with a per-name weight cap:

$$w_i = \begin{cases} \frac{e^{\ell_i}}{\sum_{j \in \mathcal{V}} e^{\ell_j}}, & i \in \mathcal{V} \\ 0, & i \notin \mathcal{V} \end{cases} \quad \text{where } \mathcal{V} = \{0\} \cup \{j : 1 \leq j \leq N, \text{mask}_j = 1\} \quad (1)$$

Cash (slot 0) is always in $\mathcal{V}$. After the softmax, equity slots that exceed the per-name cap $w_{\max} = 0.10$ are clipped, and the excess is redistributed proportionally to all uncapped slots (cash and uncapped equities) in a fixed-point iteration. Cash is never capped.

3.4 Reward Functions

Three reward functions are implemented (`src/trader/env/reward.py`):

LogReturn (current default) $r_t = \log(\text{NAV}_t / \text{NAV}_{t-1})$. Direct, dense, aligned with CAGR maximization.

ExcessLogReturn (opt-in via `use_excess_returns` flag) the env subtracts the cross-sectional mean of `log_return_1d` over tradeable stocks before passing the value to the reward function:

$$r_t = \log(\text{NAV}_t / \text{NAV}_{t-1}) - \bar{\rho}_t, \quad \bar{\rho}_t = \frac{1}{|\mathcal{V}_t|} \sum_{i \in \mathcal{V}_t} \log(p_t^{(i)} / p_{t-1}^{(i)}).$$

DifferentialSharpe (deprecated; retained for ablation) the Moody–Saffell [8] online gradient of the Sharpe ratio:

$$\delta_t = \frac{B_{t-1} \Delta A_t - \frac{1}{2} A_{t-1} \Delta B_t}{(B_{t-1} - A_{t-1}^2)^{3/2}},$$

with A, B EMAs of r and r^2 . We document in §7 why this is unsuitable for PPO with a value baseline.

A turnover penalty $-\lambda \cdot \tau_t$ is added on top, where τ_t is the L1 change in equity weights and $\lambda = 0.001$ by default.

3.5 Cost Model

ZerodhaEquityDeliveryCostModel implements the Indian equity delivery cost schedule as of 2024 (Table 5). Round-trip cost on a typical retail-size trade is approximately 15–20 bps.

Table 5: Zerodha equity delivery cost components. Rates apply to trade value in INR.

Component	Rate / Amount	Side
Brokerage	0.03%, capped at Rs. 20/order	Both
STT	0.10%	Sell only
Exchange charge (NSE)	0.00322%	Both
GST	18% on (brokerage + exchange)	Both
SEBI charges	0.0001%	Both
Stamp duty	0.015%	Buy only
DP charge	Rs. 15.93 per scrip per sell day	Sell only (per name)

3.6 Slippage Model

Fill price is the open of day t adjusted by an ATR-scaled square-root impact:

$$\hat{p}_i = p_i^{\text{open}} \cdot \left(1 + k \cdot \frac{\text{ATR}_i}{p_i^{\text{close}}} \cdot \sqrt{\frac{|\Delta q_i|}{\text{ADV20}_i}} \cdot \text{sgn}(\Delta q_i) \right), \quad k = 0.1 \quad (2)$$

where Δq_i is the change in shares of name i and ADV20 is the 20-day average dollar volume divided by close. This is a standard retail-scale impact model; institutional sizes would require a richer non-linear formula.

3.7 Episode Termination

An episode ends when either:

- $t \geq \text{episode_length}$ (truncated at 252 days, capped to fit shorter val/test panels), or
- $\text{NAV}_t < 0.10 \cdot \text{NAV}_0$ (terminated — a 90% drawdown ruin trigger).

4 Model Architecture

The policy is a three-stage pipeline (Figure 2):

1. **Per-ticker encoder** — a Temporal Convolutional Network (TCN) that collapses each ticker’s 60-day feature window into a d -dimensional embedding.
2. **Heterogeneous graph attention network** (HeteroGNN) over stocks and sectors that produces context-refined embeddings.
3. **Actor / critic heads** that consume the refined embeddings together with portfolio state.

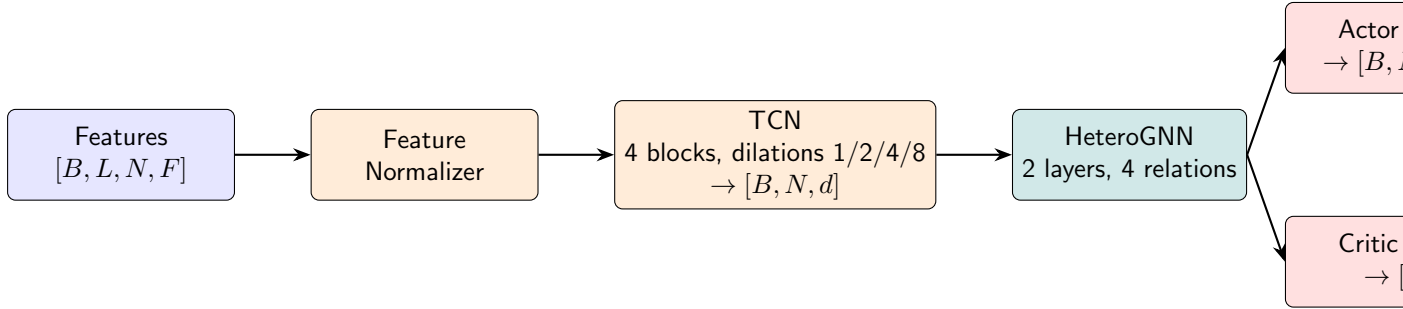


Figure 2: End-to-end policy architecture. The Feature Normalizer is a fixed (non-trainable) buffer applied at the input of the encoder; see §7 for why it is essential.

4.1 Feature Normalizer

A fixed per-feature standardization layer applied at the encoder input:

$$\tilde{x}_i = \frac{x_i - \mu_i}{\sigma_i}, \quad \mu_i, \sigma_i \text{ computed from tradeable rows of the train panel.}$$

The mean and standard deviation are computed once at training start and stored as non-trainable PyTorch buffers, so they save and load with the model. Validation and test environments use the same statistics — using their own would be a subtle form of data leakage on the policy.

This component, while trivial in isolation, was the single largest accuracy fix in the project. §7 elaborates.

4.2 TCN Encoder

A four-block causal Temporal Convolutional Network [3] with dilations $\{1, 2, 4, 8\}$. Each block contains two causal convolutions (left-padded so the output length equals the input length), LayerNorm, GELU activation, and dropout, with a residual connection. The receptive field after four blocks of kernel size 3 is

$$R = 1 + 2 \sum_{k=0}^3 2^k \cdot (3 - 1) = 31,$$

sufficient to capture intra-month structure in the 60-day window.

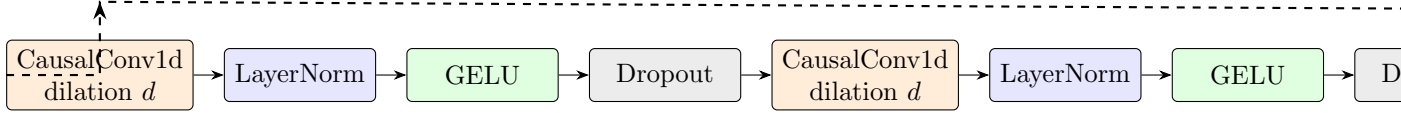


Figure 3: Single TCN block. Four such blocks are stacked with dilations $\{1, 2, 4, 8\}$.

The encoder consumes $[B, N, L, F]$ (after a permute from the env’s $[B, L, N, F]$ convention), reshapes to $[B \cdot N, F, L]$ for grouped per-ticker processing, and emits $[B, N, d]$ by taking the last timestep of the final block: $h_i = \text{TCN}(x_i)[:, :, -1] \in \mathbb{R}^d$. Weight sharing across tickers is total — there is one set of parameters for all N stocks.

4.3 Heterogeneous Graph Attention Network

The graph has two node types and four relation types:

Table 6: Node and edge types in the heterogeneous graph. $N = 163$, $S = 8$.

Node	Description
stock	One node per equity, initialized from TCN output (d -dim)
sector	One node per sector, learnable embedding (d -dim)
Relation	Edges
(stock, same_sector, stock)	All pairs (i, j) with $\sigma(i) = \sigma(j)$, bidirectional
(stock, in, sector)	$(i, \sigma(i))$ for all tradeable i
(sector, contains, stock)	Reverse of in
(sector, relates_to, sector)	All pairs of distinct sectors, bidirectional

Each relation has its own `GATv2Conv` [5] with $h = 2$ attention heads and `concat=False` (heads averaged so output dim equals input dim regardless of head count). Two stacked `HeteroConv` layers with residual connections and `LayerNorm`. `DropEdge` [7] regularization ($p=0.1$) randomly drops a fraction of edges per forward pass during training.

4.3.1 A Concrete Six-Stock Example

To make the graph structure tangible, consider a six-stock toy universe drawn from three of the eight sectors:

Tickers = (RELIANCE, ONGC, HDFCBANK, ICICIBANK, INFY, TCS)

with sector vector $\sigma = (5, 5, 1, 1, 2, 2)$ (Oil/Power, Banking, IT). Suppose all six are tradeable today, so mask = (1, 1, 1, 1, 1, 1).

The intra-sector adjacency matrix $A^{\text{intra}} \in \{0, 1\}^{6 \times 6}$ is

	REL	ONGC	HDFC	ICICI	INFY	TCS
$A^{\text{intra}} =$	REL	0	1	0	0	0
	ONGC	1	0	0	0	0
	HDFC	0	0	0	1	0
	ICICI	0	0	1	0	0
	INFY	0	0	0	0	1
	TCS	0	0	0	1	0

GATv2Conv adds self-loops for the (`stock`, `same_sector`, `stock`) relation, so the effective adjacency is $A^{\text{intra}} + I_6$.

The membership edge set is $\{(0, 4), (1, 4), (2, 0), (3, 0), (4, 1), (5, 1)\}$ (0-indexed sectors), and the inter-sector relation is the complete bidirectional graph on the three active sectors:

$$\mathcal{E}^{\text{inter}} = \{(0, 1), (1, 0), (0, 4), (4, 0), (1, 4), (4, 1)\}.$$

The full heterogeneous graph for this toy universe is illustrated in Figure 4.

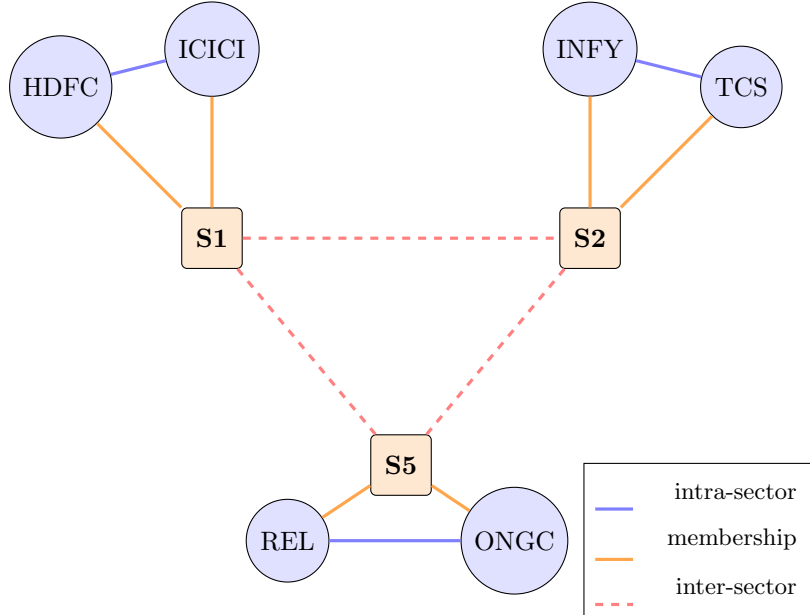


Figure 4: Heterogeneous graph for the six-stock toy universe. Round nodes are stocks (initialized from TCN output); rectangular nodes are sectors (learnable embeddings). The full graph for the 163-stock universe has 163 stock nodes, 8 sector nodes, $\sum_s \binom{|s|}{2} \cdot 2 \approx 5,500$ intra-sector edges, 326 membership edges (163 in each direction), and 56 inter-sector edges.

4.3.2 Batching and Masking

The graph topology is identical for every batch element (same universe, same sector assignments). The B subgraphs are tiled into a single disconnected graph of $B \cdot N$ stock nodes and $B \cdot S$ sector nodes, so a single `HeteroConv` call processes the entire batch.

Untradeable stocks have their input features zeroed before message passing (`stock_x = stock_x * mask.unsqueeze(-1)`) so that stale or sentinel feature values cannot leak into their neighbors' attention scores. They are also excluded from the edge index in `build_sector_edges`.

4.3.3 Ablation Variants

Three configurations exist:

- **mlp_baseline** — no graph; encoder $\rightarrow$ mean-pool $\rightarrow$ heads. The lower bound; a positive Sharpe here is a stretch goal.
- **gnn_intra_only** — only the (`stock`, `same_sector`, `stock`) relation; no sector nodes; no inter-sector edges. The midpoint between MLP and full GNN.
- **gnn_v1** — all four relations; sector nodes with learnable embeddings; the full hetero-GAT.

4.4 Actor and Critic Heads

4.4.1 Actor Head

Per-ticker logits and a cash logit are produced by two parallel MLPs:

$$p_i = \text{MLP}_{\text{eq}}([z_i, w_i^{\text{eq}}]) \in \mathbb{R}, \quad i = 1 \dots N \quad (3)$$

$$p_0 = \text{MLP}_{\text{cash}}([\bar{z}, w^{\text{cash}}]) \in \mathbb{R} \quad (4)$$

where z_i is the GNN output for stock i , $\bar{z}$ is its mean over N , and w_i are the current portfolio weights. The logit vector $\ell = (p_0, p_1, \dots, p_N)$ is the mean of a Gaussian policy with a learnable per-dimension log-std parameter:

$$a \sim \mathcal{N}(\ell, \text{diag}(e^{2 \log \sigma})).$$

Initial $\log \sigma = -1$ (so $\sigma \approx 0.37$). The masked softmax / cap projection is applied by the env, not the policy.

A critical detail (CleanRL convention [9]): the last linear layer of each MLP is initialized with orthogonal weights at gain 0.01 and zero bias. This keeps the initial logits at $\mathcal{O}(0.01)$, which after masked softmax gives a near-uniform allocation at step 0 — a stable starting point that does not blow up the trust region during the first few PPO updates.

4.4.2 Critic Head

The critic takes a richer state summary:

$$V(s) = \text{MLP}([\bar{z}, \bar{w}^{\text{eq}}, \max w^{\text{eq}}, w^{\text{cash}}, t_{\text{frac}}, r_{1\text{d}}, \sigma_{20\text{d}}, \rho_{\text{NAV}}, e^{\text{sec}}])$$

where the last block $e^{\text{sec}} \in \mathbb{R}^S$ is the per-sector portfolio exposure, computed inside the critic head via a `scatter_add` over `sector_ids` and the equity weights:

$$e_s^{\text{sec}} = \sum_{\substack{i \\ \sigma(i)=s+1}} w_i^{\text{eq}}.$$

The added scalars ($r_{1\text{d}}$, $\sigma_{20\text{d}}$, ρ_{NAV}) come from new observation keys backed by a 21-element rolling NAV history maintained in the env. This enriched state lets the critic distinguish, e.g., “+5% at low vol” from “+5% at high vol” — two states with identical NAV but very different forward expectations.

The last linear layer of the critic uses orthogonal init at gain 1.0 (standard).

4.5 Parameter Budget

A trivial parameter count for the RTX 5090. The model size has *deliberately* not been scaled up: the architecture is sample-efficient for our setting and the priority is to validate the pipeline before adding capacity.

Table 7: Approximate parameter count by component for the full `gnn_v1` configuration ($d = 64$, $h = 2$ heads, 2 GNN layers).

Component	Parameters
TCN encoder (4 blocks)	$\sim 50\text{k}$
HeteroGNN (2 layers, 4 relations)	$\sim 60\text{k}$
Sector embedding ($S \times d$)	$\sim 0.5\text{k}$
Actor head + log-std	$\sim 15\text{k} + 164$
Critic head	$\sim 15\text{k}$
Feature Normalizer (buffer, non-trainable)	30
Total trainable	$\sim 140\text{k}$

5 Reinforcement Learning Algorithm

5.1 Why PPO

We use Proximal Policy Optimization [1] for several reasons specific to this task:

1. **The action space is a constrained simplex.** A Gaussian over raw logits followed by an in-environment masked softmax composes cleanly with PPO’s clipped likelihood ratio (the Jacobian of the projection does not enter the gradient).
2. **The state is large** (~ 600 KB per step). PPO’s rollout buffer of ≤ 4096 samples is used once and discarded; an SAC-style replay buffer of 1M samples would require ~ 600 GB.
3. **Off-policy data is contaminated by the agent’s own portfolio history.** Part of s_t is the current portfolio, which is a consequence of past actions. PPO is on-policy and avoids this.
4. **The environment is essentially deterministic.** Market data is fixed; the only stochasticity is the random episode start. PPO’s “use samples once” discipline is barely a tax when env steps are cheap.
5. **Training stability dominates peak performance.** PPO is famously hard to break; SAC has more knobs (target entropy, automatic temperature, twin Q’s) and subtler failure modes. For a research project where any dial might be the cause of failure, PPO’s robustness is itself a feature.

5.2 The PPO Objective

For each minibatch the loss is

$$\mathcal{L}(\theta) = \mathbb{E}_t [\min(\rho_t A_t, \text{clip}(\rho_t, 1 - \epsilon, 1 + \epsilon) A_t) - c_v \cdot (V_\theta(s_t) - R_t)^2 + c_e \cdot H[\pi_\theta(\cdot | s_t)]] \quad (5)$$

where $\rho_t = \pi_\theta(a_t | s_t) / \pi_{\theta_{\text{old}}}(a_t | s_t)$ is the importance ratio, A_t is the GAE estimator, $R_t = A_t + V_{\theta_{\text{old}}}(s_t)$ is the bootstrapped return target, $\epsilon = 0.2$ is the clip coefficient, $c_v = 0.5$ is the value-loss weight, and $c_e = 0.001$ is the entropy bonus. The value loss is additionally clipped:

$$\mathcal{L}_v = \frac{1}{2} \max((V_\theta - R)^2, (V_{\text{old}} + \text{clip}(V_\theta - V_{\text{old}}, -\epsilon, \epsilon) - R)^2).$$

5.3 Generalized Advantage Estimation

GAE [2] is computed in reverse over the T -step rollout:

$$\delta_t = r_t + \gamma V(s_{t+1})(1 - d_t) - V(s_t) \quad (6)$$

$$A_t = \delta_t + \gamma \lambda (1 - d_t) A_{t+1} \quad (7)$$

with $\gamma = 0.995$ (matching the ~ 200 -day horizon) and $\lambda = 0.95$.

5.4 Reward Normalization

A Welford running standard deviation [10] is maintained across the entire training run; rewards are divided by it before GAE. The mean is *not* subtracted — preserving sign matters for distinguishing good vs bad trades. This keeps the gradient scale stable as the policy improves and reward magnitudes change.

5.5 Mixed-Precision Training

Forward and backward passes run under `torch.autocast(dtype=torch.bfloat16)` on CUDA. BF16’s wider exponent range avoids the gradient scaling needed for FP16, and BF16 values round-trip losslessly through FP32, so casting to numpy for env interaction is safe. A subtle bug arose when only the update used BF16 while the rollout did not (§7); the fix was to share a single `autocast` context across both phases.

5.6 Target-KL Early Stopping

After each minibatch the approximate KL divergence

$$\widehat{\text{KL}} = \mathbb{E}_t[(\rho_t - 1) - \log \rho_t]$$

is checked; if its mean over an epoch exceeds 0.02, the remaining epochs are skipped. This protects against runaway updates when the policy and the rollout distribution have drifted apart.

5.7 Algorithm Summary

6 A Step-by-Step Walkthrough

To make the system concrete, we trace one training step end to end on the toy six-stock universe from §4.3.1, with $L = 60$, $F = 15$, $d = 64$.

Step 0: Observation. The env returns

- **features:** a $60 \times 6 \times 15$ tensor; each (ℓ, n, f) entry is the feature f for stock n on day $t - 60 + \ell$.
- **mask:** $(1, 1, 1, 1, 1, 1)$.
- **sector_ids:** $(5, 5, 1, 1, 2, 2)$.
- **portfolio:** $(1, 0, 0, 0, 0, 0)$ at episode start — 100% cash.
- **t_frac:** 0.0.
- **recent_return_1d, recent_vol_20d, nav_log_progress:** 0.0 each.

Step 1: Feature normalization. Each of the 15 feature channels is standardized using buffers μ_f, σ_f computed at training start. For example, `dollar_volume_20` for RELIANCE on this day might be $8.5\text{e}9$; the normalizer maps this to $(8.5\text{e}9 - 1.29\text{e}9)/2.44\text{e}9 = 2.96$, putting it on the same numerical scale as `log_return_1d`.

Step 2: TCN encoding. The 60×15 window for each stock is processed through 4 dilated causal conv blocks. The final timestep output is a $d = 64$ vector h_i per stock — a learned summary of the past quarter.

Step 3: Hetero graph attention.

1. Stock embeddings $h \in \mathbb{R}^{6 \times 64}$ are zeroed where mask is 0 (here, none).

Algorithm 1 PPO training loop (one update iteration)

```

1: model.eval() ▷ disable DropEdge / dropout for deterministic log-probs
2: amp_ctx ← autocast(BF16, enabled = cuda)
3: rollout buffers ← ∅
4: with torch.no_grad(), amp_ctx:
5:   for  $t = 1 \dots T$  do
6:     Sample action  $a_t$  from  $\pi_{\theta_{\text{old}}}(\cdot | s_t)$ 
7:     Step env:  $(s_{t+1}, r_t, d_t) \leftarrow \text{env}(s_t, a_t)$ 
8:     Append  $(s_t, a_t, \log \pi(a_t), V(s_t), r_t, d_t)$  to buffer
9:   end for
10: Bootstrap  $V(s_T)$  for last state
11: if normalize_rewards then
12:   Update  $\hat{\sigma}_r$ ;  $r_t \leftarrow r_t / \hat{\sigma}_r$ 
13: end if
14: Compute  $A_t, R_t$  via GAE
15: Anneal LR:  $\eta \leftarrow \eta_0 \cdot (1 - \text{update}/\text{n\_updates})$ 
16: for epoch =  $1 \dots E$  do
17:   Shuffle and split into  $M$  minibatches
18:   for each minibatch do
19:     with amp_ctx: compute  $\log \pi_\theta(a), H[\pi_\theta], V_\theta(s)$ 
20:     Normalize advantages:  $A \leftarrow (A - \bar{A}) / \sigma_A$ 
21:     Compute  $\mathcal{L}$  per Eq. (5)
22:     Backprop, clip grad norm, Adam step
23:   end for
24:   if  $\overline{\text{KL}} > 0.02$  break ▷ target-KL early stop
25: end for

```

2. Sector embeddings are looked up: $E^{\text{sec}} \in \mathbb{R}^{8 \times 64}$.

3. Edges are constructed (Figure 4) and DropEdge is applied (eval mode in rollout, so a no-op). The edge dictionary is tiled across the batch.

4. One HeteroConv call processes all four relations:

$$h_i^{(1)} = \text{LayerNorm}\left(\text{GELU}(\text{HeteroAgg}(h, E^{\text{sec}}, \mathcal{E})_i) + h_i\right) \quad (8)$$

$$E_s^{\text{sec}, (1)} = \text{LayerNorm}\left(\text{GELU}(\dots) + E_s^{\text{sec}}\right) \quad (9)$$

5. A second layer is applied analogously, producing the final stock representation $z \in \mathbb{R}^{6 \times 64}$.

Step 4: Actor head. For each stock i , $p_i = \text{MLP}_{\text{eq}}([z_i, w_i^{\text{eq}}])$. At init (small-init weights), all $p_i \approx 0$ and similarly $p_0 \approx 0$. The Gaussian policy samples

$$a \sim \mathcal{N}(\ell, \sigma^2 I), \quad \ell \approx \mathbf{0}, \quad \sigma \approx 0.37,$$

yielding raw logits roughly in $[-0.7, +0.7]$.

Step 5: Critic head. A scalar value

$$V(s) = \text{MLP}([\bar{z}, \text{summary scalars (7), sector exposures (8)}])$$

is computed. At init it is also near zero.

Step 6: Action projection in env. The masked softmax with cap is applied: $w = \text{masked_softmax}(a, \text{mask}, w_{\text{max}} = 0.10)$. At init with $\sigma = 0.37$, sample logits give w roughly

uniform around $1/(N + 1) \approx 0.14$ on the toy universe (or ≈ 0.006 per slot on the full 163-stock universe before the cap kicks in).

Step 7: Trade execution. Target shares are computed at the open with floor rounding; slippage is applied; costs are deducted; the new position is marked to close. NAV update gives $\log r_t = \log(\text{NAV}_t / \text{NAV}_{t-1})$.

Step 8: Reward. With the default LogReturn reward and turnover penalty $\lambda = 0.001$,

$$r_t = \log r_t - \lambda \tau_t.$$

Step 9: Buffer. The full $(s_t, a_t, \log \pi(a_t), V(s_t), r_t, d_t)$ tuple is appended to the rollout buffer. After $T = 256$ such steps across each of $E = 16$ envs, the PPO update consumes the resulting 4096-sample batch.

7 The Debugging Journey

This section is included not for completeness but because the debugging arc *is* the result. Each bug masqueraded as a different one; fixing the wrong one first would have produced no improvement and led us to abandon a working architecture.

7.1 Run-by-Run Status

Table 8: Training run history. “r5” refers to the fifth restart of the GNN runs prior to identifying the BF16/FP32 precision bug; “r6” is the current run with all known fixes. Final figures are training-set Sharpe over the last 10 episodes.

Run	Code state	Train Sharpe	Val Sharpe	Notes
r1–r4	DSR + train()-mode rollout	diverged	—	GNN clip_frac=1, KL=36–161
r5 MLP	DSR + eval()	−1.33	−2.09	Stable, uniform policy
r5 GNN_v1	DSR + BF16/FP32 fix	−1.39	—	Killed pre-completion
r5 GNN_intra	DSR + BF16/FP32 fix	−1.89	—	Killed pre-completion
r6 MLP	LogReturn + feat-norm + crit-enrich + actor-init	−1.12	−1.66	Better than r5
r6 GNN_v1	+ above	−2.36	—	At 32% — concerning
r6 GNN_intra	+ above	+0.031	—	At 44% — first positive!

7.2 Bug 1: Stochastic Operations During Rollout

Symptom. GNN runs diverged with `clip_frac = 1.000` and `approx_kl = 36--161` from the very first update. MLP runs were unaffected.

Diagnosis. `torch.no_grad()` disables gradient tracking but does *not* disable dropout or any other stochastic operation. With the model in `train()` mode during rollout, both `DropEdge` (random edge removal, $p = 0.1$) and `GATv2Conv`’s attention dropout fire. The rollout’s $\log \pi_{\theta_{\text{old}}}(a)$ was computed under one random graph topology; the update’s $\log \pi_{\theta}(a)$ was computed under a *different* topology. The importance ratio $\exp(\log \pi_{\theta} - \log \pi_{\theta_{\text{old}}})$ was measuring random topology change, not weight change.

The MLP was less affected only because TCN dropout has smaller functional impact than two layers of graph attention plus `DropEdge`.

Fix. Call `self.model.eval()` before the rollout and restore `self.model.train()` only at the end of the training loop. *Forgetting this is a silent killer for any RL policy with stochastic operations in the forward path.*

7.3 Bug 2: BF16 / FP32 Precision Mismatch

Symptom. Even after Bug 1 was fixed, GNN runs showed `clip_frac = 0.978` on the very first PPO update — before any gradient step had been applied. Identical pre-update weights were producing different log-probs in rollout vs update.

Diagnosis. The rollout was running in plain FP32 (no autocast); the update was wrapped in BF16 autocast. BF16 has 7 mantissa bits versus FP32’s 23, giving roughly 0.8% relative error per matmul. Over two `GATv2Conv` layers (each with multiple matmuls per attention head), the error compounds to several percent, which is enough to shift logits by $O(0.1)$ and explode the KL.

Severity correlated with depth. GNN_v1 (4 edge types, more matmuls) showed KL = 161; GNN_intra (1 edge type) showed KL = 50.8; MLP (no graph) showed `clip_frac = 0.065` — mild but present.

Fix. Create the `autocast` context *before* the rollout and use it in both with `torch.no_grad()`, `amp_ctx:` and with `amp_ctx:` update blocks. Cast model outputs to FP32 before `.cpu().numpy()` (BF16 is not a numpy dtype).

7.4 Bug 3: DifferentialSharpe Reward Has Zero Expected Value

Symptom. After Bugs 1 and 2 were fixed, the MLP trained stably to Sharpe -1.33 over 8M steps. The agent did not catastrophically fail, but it also did not learn to trade. The policy converged to a high-entropy, near-uniform allocation. CAGR was -16.6% in a market where the equal-weight baseline returns $+16.4\%$ (a 33-percentage-point gap).

Diagnosis. The DSR formula

$$\delta_t = \frac{B_{t-1}(r_t - A_{t-1}) - \frac{1}{2}A_{t-1}(r_t^2 - B_{t-1})}{(B - A^2)^{3/2}}$$

at steady state simplifies to $\delta_t \approx C \cdot (r_t - \mu_t)$ where μ_t is the running EMA mean of returns. Its expected value is therefore *zero*, regardless of whether μ_t is positive (good portfolio) or negative (bad portfolio). The PPO value head learned $V(s) \approx 0$ for all s ; advantages became pure noise; the only remaining gradient was from the entropy bonus ($c_e = 0.001$), which slowly randomized the policy.

DSR was designed for direct policy gradient (REINFORCE) where its zero-mean property is fine because there is no value baseline. With a critic in PPO it fails.

Fix. Switch to plain $r_t = \log(\text{NAV}_t / \text{NAV}_{t-1})$ (LogReturn). Bonus bug found in passing: the YAML key `reward: differential_sharpe` was *never read* by `train.py` — the env always used the default `DifferentialSharpe` regardless of config. We wired up a `_REWARD_MAP` in `train.py`.

7.5 Bug 4: Feature Scale Catastrophe

Symptom. Even after switching to LogReturn, the MLP made only modest gains and the agent still allocated near-randomly. The model was trained but shouldn’t have been — feature scales differed by eleven orders of magnitude.

Diagnosis. See the std column of Table 3. Without input normalization, the TCN’s first conv layer applies $\mathbf{W}x \in \mathbb{R}^{C \times L}$ with $\mathbf{W}$ at Xavier scale $\mathcal{O}(1/\sqrt{F})$. The pre-LayerNorm output for `dollar_volume_20` is $\sim 10^9 \cdot 0.25 = 2.5 \times 10^8$; for `log_return_1d` it is $\sim 10^{-2} \cdot 0.25 = 2.5 \times 10^{-3}$. After LayerNorm the dollar-volume signal dominates the entire normalized output and the small-scale signals (returns, RSI, momentum, vol) are reduced to numerical noise. The model has effectively been allocating based on a noisy ranking of 20-day dollar volume.

Fix. A fixed `FeatureNormalizer` layer at the encoder input, applying $(x - \mu)/\sigma$ where μ, σ are computed from the *tradeable* rows of the train panel. Stats are saved to `data/panels/train.feature.stats` alongside the panel SHA256.

This fix alone moved MLP train Sharpe from -1.33 to -1.12 and enabled GNN_intra to reach the first positive Sharpe of the project.

7.6 Bug 5: Environment Was the Wall-Clock Bottleneck

Symptom. 8M steps took roughly 12 hours on the RTX 5090 at ~ 185 SPS across 16 envs — around 11 SPS per env. GPU utilization was 15–20%.

Diagnosis. `PanelTradingEnv.build_obs()` contained a double Python for-loop over (features, tickers) with dict lookups: $60 \times 163 \times 15 = 146,700$ Python operations per step. `_price_at()`, `_mask_at()`, `_sector_ids_at()` each did 163 dict lookups per call. The cost calculation in `step()` was another per-ticker Python loop over the 163-name universe. The GPU was starved waiting for the env.

Fix. At env construction:

- Pre-stack features to a single $[T, N, F]$ NumPy array.
- Pre-stack each price/metadata column to $[T, N]$.
- Pre-stack the tradeable mask to $[T, N]$ bool.

Then `_build_obs()` is a single slice operation; `_price_at()` is a single index; the cost loop is replaced by a vectorized `cost_vec()` method on the cost model that uses pure NumPy.

Measured impact. Single-env step rate jumped from ~ 11 SPS to $\sim 12,000$ SPS — roughly $1,000\times$ speedup on the env alone. End-to-end training is now GPU-bound; r6 MLP achieved 254 SPS (37% above the r5 baseline of 185 SPS) and r6 GNN_intra is at 142 SPS.

7.7 Architectural Hardening (Beyond Bugs)

Two additional improvements were applied in the same r6 pass:

Critic enrichment. The critic head was given access to per-sector exposure (8 dims via `scatter_add`) and three new portfolio scalars (recent return, recent vol, NAV log progress). Lower critic variance $\Rightarrow$ lower advantage variance $\Rightarrow$ more stable PPO updates.

Small-init actor. Both actor MLPs' final linear layers are initialized with orthogonal weights at gain 0.01 (CleanRL convention). The pre-softmax logits at step 0 are bounded by ~ 0.018 , giving a near-uniform allocation. This is the standard cure for early-training KL spikes in PPO.

8 Empirical Results

8.1 Baselines

The baseline reference is `equal_weight_rebalanced` on the train panel: **Sharpe = 1.04**, **CAGR = 16.4%**, **vol = 15.7%**. This is the bar the RL agent must beat on *validation* for M5/M6 acceptance.

8.2 r6 Results In Progress

The most encouraging signal is `rew_std` = 0.010 across all three runs — exactly the LogReturn scale (daily log return std on a diversified Indian equity portfolio is $\sim 1\%$). With the previous DSR reward this metric was 4–7. The fact that all three runs show `clip_frac` $\in [0.30, 0.32]$ is a textbook healthy band.

Table 9: Current r6 metrics as of April 26, 2026, mid-run. The MLP completed training; the two GNN runs are in progress.

Run	Update	Train Sharpe	Val Sharpe	CAGR	Clip frac	SPS
mlp_baseline	1953/1953	-1.12	-1.66	-13.8%	0.004	254
gnn.v1	630/1953	-2.36	—	-30.7%	0.298	104
gnn.intra_only	860/1953	+0.031	—	+7.3%	0.324	142

8.3 Why GNN_v1 Underperforms GNN_intra

A surprising and currently un-explained result: the simpler intra-only graph materially outperforms the full hetero-GNN through the first $\sim 30\%$ of training. A working hypothesis (to be confirmed in r7): PyTorch’s default `nn.Embedding` initialization is $\mathcal{N}(0, 1)$, but the LayerNorm-ed TCN output has unit norm. The sector embeddings are therefore $\sim 8\times$ larger than the stock embeddings entering the GNN, and via the (`sector`, `contains`, `stock`) relation they dominate the message passing for hundreds of updates. The intra-only variant has no sector nodes and is immune.

The fix (one line) is queued for r7:

```
1 nn.init.normal_(self.sector_emb.weight, mean=0.0, std=0.1)
```

9 Hyperparameter Reference

9.1 PPO Hyperparameters

Table 10: PPO hyperparameter cheat-sheet — what each knob does and the empirical range that produced healthy training in r6.

Param	Value	Effect
<code>learning_rate</code>	3e-4 (MLP) / 5e-5 (GNN)	GNN attention layers have larger gradient norms; smaller LR prevents trust-region exits
<code>clip_coef</code> (ϵ)	0.2	PPO ratio clipping threshold; 0.2 is canonical
<code>ent_coef</code> (c_e)	0.001	Lower than PPO defaults (0.01) — at 0.01 entropy dominated the gradient signal in this task
<code>vf_coef</code> (c_v)	0.5	Standard
<code>max_grad_norm</code>	0.5 (MLP) / 0.2 (GNN)	GNN grad norms are larger; tighter clipping required
<code>gamma</code> (γ)	0.995	~ 200 -day effective horizon (vs 252-day episode)
<code>gae_lambda</code> (λ)	0.95	Standard bias/variance tradeoff
<code>n_steps</code>	256	Rollout length per env
<code>n_envs</code>	16	SyncVectorEnv worker count
<code>n_minibatches</code>	32	32, not 64 — 64 caused excessive sequential drift in r4
<code>n_epochs</code>	4	Standard; combined with <code>target_kl</code> early stopping
<code>target_kl</code>	0.02	Threshold for early-stopping the epoch loop
<code>normalize_advantage</code>	true	Per-minibatch zero-mean unit-std
<code>normalize_rewards</code>	true	Welford running std, mean NOT subtracted
<code>anneal_lr</code>	true	Linear decay to 0 at final update

Table 11: Model hyperparameters and the components they affect.

Param	Value	Affects
embed_dim (d)	64	TCN output, GNN node dim, head input
kernel_size	3	TCN receptive field $\propto$ kernel
num_channels	[64, 64, 64, 64]	TCN depth and width
dropout (TCN)	0.1	Generalization; only active in train() mode
num_layers (GNN)	2	Message-passing depth; k -hop neighborhood
num_heads (GAT)	2	Attention diversity
dropout (GAT)	0.1	Attention regularization
drop_edge_prob	0.1	DropEdge regularization; only active in train()
relations	all / intra_only	Ablation control
num_sectors	8	Critic exposure dim, sector embedding count
head_hidden	64	Width of actor / critic MLPs
log_std_init	-1 ($\sigma \approx 0.37$)	Initial Gaussian policy std

9.2 Model Hyperparameters

9.3 Environment Hyperparameters

Table 12: Environment hyperparameters.

Param	Value	Affects
lookback_days	60	Feature window length L
episode_length	252	Trading days per episode (auto-clamped on shorter panels)
initial_cash	Rs. 1,000,000	Realistic retail account size for NSE
max_weight_per_name	0.10	Diversification floor; min ~ 10 names if fully invested
turnover_penalty	0.001	L1 weight-change penalty in reward
reward	log_return	vs differential_sharpe / excess_log_return
use_excess_returns	false	Subtract benchmark before reward (opt-in)

10 Future Work

10.1 r7: Pre-staged Improvements

The following changes are already implemented and gated as opt-in (so they did not perturb the in-flight r6 runs):

1. **Sector embedding init at $\mathcal{N}(0, 0.1)$.** Active on fresh runs only.
2. **ExcessLogReturn reward + use_excess_returns env flag.** Subtracts the cross-sectional mean log return from the portfolio log return before reward computation, giving the agent a direct incentive to beat the universe average rather than just earn positive returns.
3. **Lower ent_coef to 1e-4.** End-of-r6 diagnostics show σ drifting from 0.37 to ~ 0.65 over training. A $10\times$ smaller entropy bonus will tighten the policy in late training without hurting early exploration.
4. **Per-batch tradeable mask in GNN edge construction.** Currently `mask[0]` is reused for every batch element. Untradeable stocks are correctly zeroed in features but still appear as graph nodes with messaging contributions. Deferred until results indicate it matters.

10.2 M7: Walk-Forward Training

`src/trader/training/walk_forward.py` (not yet implemented) will run **4 windows** of (24-month train, 1-month purge, 6-month val, 6-month test) sliding through the full panel, with **5 seeds** per window. A paired bootstrap CI of RL agent vs `equal_weight` on each window is the acceptance test. A shuffled-ticker-label sanity check is wired behind a flag to detect spurious signal exploitation.

10.3 M8–M10: Paper Broker, Live Stub, End-to-End Report

The remaining three milestones are scoped:

- **M8:** Paper broker backed by Postgres ledger; T+1 cash settlement; golden-file integration test (fixed seed → fixed fills).
- **M9:** Zerodha Kite Connect adapter stub (interface only, `NotImplementedError` bodies).
- **M10:** `full_report.py` producing a single PDF with equity curves, drawdown, per-sector allocation over time, GNN attention heatmap, and paired bootstrap significance tables.

10.4 Beyond M10: Possible Extensions

Only after M10 (a complete walk-forward baseline that beats `equal_weight`) is in hand are the following worth considering:

- Sentiment embeddings (offline; per-(ticker, week) static representations).
- Hierarchical two-level policy: weekly sector allocation (slow) + daily within-sector picks (fast).
- Intraday data and a new env variant (5-minute bars).
- Soft Actor-Critic ablation (after PPO has been proven sufficient).
- Live adapter (separate stream with its own risk review).

11 Lessons Learned

A few practical conclusions earned the hard way during M5–M6:

1. **Algorithm choice rarely fixes a broken environment, reward, or feature pipeline.** We considered switching to SAC mid-debug; doing so would have hidden Bug 4 (feature normalization) for weeks and confounded the diagnosis with SAC’s own tuning subtleties. The discipline of fixing the environment first paid off.
2. **The stochastic-rollout bug is universal for graph or high-dropout policies.** `torch.no_grad()` is not a substitute for `model.eval()`. Any RL pipeline that uses DropEdge, attention dropout, or batch-norm-style stochastic operations should call `eval()` during rollout.
3. **Reward functions for PPO must have non-zero expected value at optimum.** The Differential Sharpe is mathematically elegant but unsuited for an algorithm with a value baseline. Reward design and algorithm design are coupled.
4. **Feature scale standardization is the cheapest large-effect fix in deep RL.** A single `FeatureNormalizer` buffer moved a -1.33 Sharpe agent into the regime where actual learning is possible.

5. **Profile the env before assuming the GPU is the bottleneck.** A $1,000\times$ env speedup turned a CPU-bound 12-hour run into a GPU-bound 3–5-hour run with no model changes.
6. **Logs are not enough; instrument early.** `clip_frac`, `approx_kl`, and `rew_std` caught bugs that final-Sharpe alone would have hidden behind seed variance.

References

- [1] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov. *Proximal Policy Optimization Algorithms*. arXiv:1707.06347, 2017.
- [2] J. Schulman, P. Moritz, S. Levine, M. Jordan, P. Abbeel. *High-Dimensional Continuous Control Using Generalized Advantage Estimation*. ICLR, 2016.
- [3] S. Bai, J. Z. Kolter, V. Koltun. *An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling*. arXiv:1803.01271, 2018.
- [4] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, Y. Bengio. *Graph Attention Networks*. ICLR, 2018.
- [5] S. Brody, U. Alon, E. Yahav. *How Attentive are Graph Attention Networks?* ICLR, 2022.
- [6] M. Schlichtkrull, T. N. Kipf, P. Bloem, R. van den Berg, I. Titov, M. Welling. *Modeling Relational Data with Graph Convolutional Networks*. ESWC, 2018.
- [7] Y. Rong, W. Huang, T. Xu, J. Huang. *DropEdge: Towards Deep Graph Convolutional Networks on Node Classification*. ICLR, 2020.
- [8] J. Moody, M. Saffell. *Learning to Trade via Direct Reinforcement*. IEEE Transactions on Neural Networks, 12(4), 2001.
- [9] S. Huang, R. F. J. Dossa, C. Ye, J. Braga, D. Chakraborty, K. Mehta, J. G. Araújo. *CleanRL: High-quality Single-file Implementations of Deep Reinforcement Learning Algorithms*. JMLR, 2022.
- [10] B. P. Welford. *Note on a Method for Calculating Corrected Sums of Squares and Products*. Technometrics, 4(3), 1962.
- [11] T. Haarnoja, A. Zhou, P. Abbeel, S. Levine. *Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor*. ICML, 2018.
- [12] X.-Y. Liu et al. *FinRL: Deep Reinforcement Learning Framework to Automate Trading in Quantitative Finance*. ACM ICAIF, 2022.
- [13] J. Moody, L. Wu, Y. Liao, M. Saffell. *Performance Functions and Reinforcement Learning for Trading Systems and Portfolios*. Journal of Forecasting, 17, 1998.